

Week 8 Module

1. Fraud ^{transaction} detection (Linear Search)

Problem statement :-

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists. WAP using linear search.

Sol: ① Approach:-

- We are given N transaction IDs in unsorted order.
- We traverse the array from index 0 to $N-1$, one element at a time.
- At each index, we compare the current element with the target (suspicious ID).
- If a match is found, we print "Fraud Transaction Found" and stop.
- If we reach the end of the array without finding a match, we print "Transaction Not Found."

Key Point:-

Since the array is unsorted, we cannot use Binary search. Linear search is the only option here.

Time complexity :- $O(N)$

space complexity :- $O(1)$

2. Search log search (Binary Search)

Problem Statement:-

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

WAP a program using Binary Search.

Sol:- Approach:-

- we are given N log IDs in sorted order.
- we maintain three pointers - $low = 0$, $high = N - 1$, and $mid = (low + high) / 2$.
- At each step, we compare $arr[mid]$ with the target.
 - If $arr[mid] == target \rightarrow$ element found, print "log found" and stop.
 - If $arr[mid] < target \rightarrow$ target is in the right half, so set $low = mid + 1$.
 - If $arr[mid] > target \rightarrow$ target is in the left half, so set $high = mid - 1$.
- We repeat this until $low > high$.
- If loop ends without finding, print "log not found."

Key Point:- Binary Search only works on sorted arrays. Each step reduces the search space by half.

Time complexity: $O(\log N)$

Space complexity: $O(1)$

3. Sort Employee Salaries (Bubble Sort)

Problem Statement:-

A company wants to sort employee salaries in ascending order for payroll analysis.

WAP using Bubble Sort.

soln Approach:-

- We are given N salaries in unsorted order. We need to sort them in ascending order.
- We run $N-1$ outer passes over the array.
- In each pass, we compare adjacent elements $arr[i]$ and $arr[i+1]$.
- If $arr[i] > arr[i+1]$, we swap them.
- After each pass, the largest unsorted element "bubbles up" to its correct position at the end.
- In the next pass, we don't need to check the last already-sorted element, so inner loop runs up to $N-1-i$.

Key Point:- After every pass, at least one element gets placed at its correct final position.

Time complexity:- $O(N^2)$

Space complexity:- $O(1)$

4. sort website Response Time (Insertion sort)

Problem statement:-

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. WAP using insertion sort.

Sol:- Approach:-

- We are given N response times, we need to sort them in ascending order.
- We consider the first element as already sorted. We start from index 1.
- For each element at index i , we store it as $key = arr[i]$.
- We then compare key with all elements in the sorted portion (index 0 to $i-1$) going from right to left.
- Every element that is greater than key is shifted one position to the right.
- Once we find an element smaller than or equal to key (or reach the start), we place key at that position.
- We repeat this for all elements from index 1 to $N-1$.

Key Point:- At any point, the left portion of the array (0 to i) is always sorted. It will

like inserting a card in the correct position in a hand of cards.

Time complexity: $O(N^2)$ Space complexity: $O(1)$

5. Prioritylike Bug Reports (selection sort)

Problem Statement:-

A software company assigns priority scores to bug reports. To resolve bugs effectively, they want to sort priorities in ascending order. WAP a program using selection sort.

Approach:-

- We are given N priority scores. We need to sort them in ascending order.
- We run $N-1$ outer passes. In each pass, we find the minimum element from the unsorted portion of the array.
- We assume the first element of the unsorted portion is the minimum ($\text{minIdx} = i$).
- We scan from $i+1$ to $N-1$. If we find any element smaller than $\text{arr}[\text{minIdx}]$, we update minIdx .
- After the inner loop, we swap $\text{arr}[i]$ with $\text{arr}[\text{minIdx}]$.
- This places the smallest unsorted element at its correct position.

- we repeat until the array is fully sorted.

Key point: - In each pass, exactly one element is selected and placed at its correct position. Unlike Bubble sort, Selection sort does fewer swaps (at most $N-1$ swaps total).

Time complexity :- $O(N^2)$

Space complexity :- $O(1)$